Computer Exercise 1: Worked solutions

Algorithms, Programming, and Data Representation · Module 92313 · Exercise 1

From Fundamentals to AI Apocalypse · personal study guide

Contents

Task 1: Read only the requested prefix 2

Task 2: Create a useful signature table 2

Task 3: Identify a type and resolve ZIP subtypes 2

Task 4: Hash a file and verify integrity 2

Task 5: Find mismatched file extensions recursively 3

Task 6: Find files that validly match multiple formats 3

Task 7: Format an exact hexdump 3

Task 8: Classify common text encodings from bytes 3

Task 9: Find the first byte after the valid file structure 4

Task 10: Aggregate the forensic report and write the personalized justification 4

Answers 5

Made by Thanate Amornjiraporn (Focus)

File signature forensics works from bytes and validated structures, never from filenames alone. Try each task before opening its solution. The ZIP family and the personalized sample offsets need the extra care explained below.

Task 1: Read only the requested prefix

Connected topic: Computers, programs, and data representation (Chapter 2)

1 · read_magic_bytes(filepath, n=16)

Try first: decide what the function must return for a short file, an empty file, and n=0 before writing it.

Answers: see "Task 1: Read only the requested prefix" under Answers at the end.

Task 2: Create a useful signature table

Connected topic: Integer representations and binary data (§3.2)

2 · SIGNATURES

Try first: list real leading byte patterns for distinct formats, then separate container formats from their packaged subtypes.

Answers: see "Task 2: Create a useful signature table" under Answers at the end.

Task 3: Identify a type and resolve ZIP subtypes

Connected topic: Integer representations and binary data (§3.2)

3 · identify_file_type(magic)

Try first: separate "what the first bytes suggest" from "what the whole container contains."

Answers: see "Task 3: Identify a type and resolve ZIP subtypes" under Answers at the end.

Task 4: Hash a file and verify integrity

Connected topic: Computers, programs, and data representation (Chapter 2)

4 · compute_checksum / verify_integrity

Try first: process the file in fixed-size chunks so memory use does not grow with the file size.

Answers: see "Task 4: Hash a file and verify integrity" under Answers at the end.

Made by Thanate Anorrijiraporn (Focus)

Task 5: Find mismatched file extensions recursively

Connected topic: Computers, programs, and data representation (Chapter 2)

5 · find_mismatched_extensions(root_dir)

Try first: compare the suffix claim with the type discovered from the file bytes and, for ZIPs, its member list.

Answers: see "Task 5: Find mismatched file extensions recursively" under Answers at the end.

Task 6: Find files that validly match multiple formats

Connected topic: Integer representations and binary data (§3.2)

6 · detect_polyglot(filepath)

Try first: collect and validate candidates independently instead of returning the first signature hit.

Answers: see "Task 6: Find files that validly match multiple formats" under Answers at the end.

Task 7: Format an exact hexdump

Connected topic: Character and byte representations (§3.3)

7 · bytes_to_hexdump(data, width=16)

Try first: work from chunks so offsets and columns are stable even on the last, partial line.

Answers: see "Task 7: Format an exact hexdump" under Answers at the end.

Task 8: Classify common text encodings from bytes

Connected topic: Character and byte representations (§3.3)

8 · detect_text_encoding(filepath)

Try first: check BOMs first, then use byte patterns and a strict UTF-8 validity check.

Answers: see "Task 8: Classify common text encodings from bytes" under Answers at the end.

Made by Thanate Angornjirapern (Focus)

Task 9: Find the first byte after the valid file structure

Connected topic: Integer representations and binary data (§3.2)

9 · find_appended_data(filepath, filetype)

Try first: parse the end of the format structure and compare that offset with the full file length.

Answers: see "Task 9: Find the first byte after the valid file structure" under Answers at the end.

Task 10: Aggregate the forensic report and write the personalized justification

Connected topic: Computers, programs, and data representation (Chapter 2)

10 · generate_forensics_report(root_dir)

Try first: reuse one read per file, make a JSON-safe record, then measure the runtime on 500 or more files.

Answers: see "Task 10: Aggregate the forensic report and write the personalized justification" under Answers at the end.

Written justification · use your own sample offsets

Try first: record the polyglot file and appended-data boundary from your assigned samples; the seeded filenames and offsets differ by learner.

Answers: see "Task 10: Aggregate the forensic report and write the personalized justification" under Answers at the end.

Made by Thanate Amornjirapong (Focus)

Answers

Task 1: Read only the requested prefix

1 · read_magic_bytes(filepath, n=16) · worked solution

1. The input is a path to an arbitrary binary file. Open it in binary mode so the bytes are not decoded or translated. The result is the first n bytes, not the whole file.
2. A bounded read naturally returns fewer than n bytes at end-of-file. Reject a negative n: Python treats `read(-1)` as "read the rest", which would break the memory limit.
3. Close the handle on success and on exceptions with a context manager. Check a short file and n=0: the returned values should be the bytes available and `b""`, respectively.
4. **The whole function:**

```
def read_magic_bytes(filepath, n=16):
  if n < 0:
      raise ValueError("n must be nonnegative")
  with open(filepath, "rb") as f:
      return f.read(n)
```

Made by Thanate Amornj

Task 2: Create a useful signature table

2 · SIGNATURES · worked solution

1. A signature is bytes, so store bytes literals and match from offset zero. Include multiple common JPEG starts and both GIF header versions. Add at least twelve distinct formats.
2. DOCX, XLSX, PPTX, and JAR are ZIP containers: their local-file headers share the same PK prefix. Repeating that one prefix under four names does not distinguish them. Add these names as archive subtypes in a second-stage classifier, not as unique magic signatures.
3. The sample table below covers thirteen signatures. It is a candidate table; a matching prefix is evidence to validate the format structure, not a complete proof that the file is valid.
4. **The table:**

```
SIGNATURES = {
  "PNG": (b"\x89PNG\r\n\x1a\n",),
  "JPEG": (b"\xff\xd8\xff\xe0", b"\xff\xd8\xff\xe1", b"\xff\xd8\xff\xdb"),
  "GIF": (b"GIF87a", b"GIF89a"),
  "PDF": (b"%PDF-",),
  "ZIP": (b"PK\x03\x04", b"PK\x05\x06", b"PK\x07\x08"),
  "GZIP": (b"\x1f\x8b",),
  "ELF": (b"\x7fELF",),
  "RTF": (b"{\\rtf",),
  "BMP": (b"BM",),
  "SQLite": (b"SQLite format 3\x00",),
  "7z": (b"7z\xbc\xaf\x27\x1c",),
  "Ogg": (b"OggS",),
  "MP3": (b"ID3",),
}
```

Made by Thanat

Task 3: Identify a type and resolve ZIP subtypes

3 · identify_file_type(magic) · worked solution

1. For ordinary types, compare the available prefix to each signature; `startswith` is safe for short byte strings and cannot index past the end. If several patterns match, prefer the longest matching signature.
2. The sheet's request to distinguish DOCX/XLSX/PPTX/JAR from magic alone is underspecified: their first bytes are shared. With only the first 16 bytes, the correct answer is ZIP. To refine it, inspect the archive directory: `word/document.xml` indicates DOCX, `xl/workbook.xml` XLSX, `ppt/presentation.xml` PPTX, and `META-INF/MANIFEST.MF` JAR.
3. Use "unknown" only when no signature matches. Keep the container decision explicit so a generic ZIP is not mislabeled as an office document.
4. **The functions** (with `import zipfile` at the top of the file):

```
def identify_file_type(magic):
  matches = [(len(sig), kind)
            for kind, signatures in SIGNATURES.items()
            for sig in signatures if magic.startswith(sig)]
  return max(matches)[1] if matches else "unknown"

def identify_zip_members(source):
  with zipfile.ZipFile(source) as archive:
      names = set(archive.namelist())
  if "word/document.xml" in names:
      return "DOCX"
  if "xl/workbook.xml" in names:
      return "XLSX"
  if "ppt/presentation.xml" in names:
      return "PPTX"
  if "META-INF/MANIFEST.MF" in names:
      return "JAR"
  return "ZIP"

def classify_path(path):
  magic = read_magic_bytes(path, n=4096)
  kind = identify_file_type(magic)
  if kind == "ZIP":
      return identify_zip_members(path) if zipfile.is_zipfile(path) else "unknown"
  return kind
```

Task 4: Hash a file and verify integrity

4 · compute_checksum / verify_integrity · worked solution

1. Create the requested hashlib digest by algorithm name. Unsupported names should raise the documented `ValueError` rather than silently switching algorithms.
2. Read a chunk, update the digest, and repeat until an empty bytes value signals EOF. `hexdigest` gives the stable printable representation.
3. Verification means "recompute and compare with the expected digest"; a match detects accidental or malicious changes only when the expected digest is trusted. It does not prove who created the file.
4. **Both functions** (with `import hashlib, hmac`):

```
def compute_checksum(filepath, algo="sha256"):
  digest = hashlib.new(algo)
  with open(filepath, "rb") as f:
      for chunk in iter(lambda: f.read(65536), b""):
          digest.update(chunk)
  return digest.hexdigest()

def verify_integrity(filepath, expected, algo="sha256"):
  actual = compute_checksum(filepath, algo)
  return hmac.compare_digest(actual, expected.strip().lower())
```

Made by Thanate Amc

Task 5: Find mismatched file extensions recursively

5 · find_mismatched_extensions(root_dir) · worked solution

1. `os.walk` visits nested directories without a fixed depth. Normalize the suffix to lowercase and use a table; keep an unrecognized extension as "unknown" rather than guessing.
2. Classify content independently of the suffix. For a ZIP signature, open the archive and use Task 3's member-name check to distinguish a plain ZIP from an office package or JAR.
3. Only after a signature lookup returns unknown, classify text from the bytes; this must work for a text file with any suffix. Then compare the independently detected actual type with the suffix claim. Test a nested wrong extension, uppercase suffix, missing suffix, and unknown actual signature.
4. **The function.** It uses `EXTENSION_TYPES`, a dictionary from lowercase suffix to type name such as `{".png": "PNG", ".txt": "TXT"}`, and the helpers from Tasks 3 and 8:

```
def find_mismatched_extensions(root_dir):
  mismatches = []
  for folder, _, filenames in os.walk(root_dir):
      for filename in filenames:
          path = os.path.join(folder, filename)
          extension = os.path.splitext(filename)[1].lower()
          claimed = EXTENSION_TYPES.get(extension, "unknown")
          actual = classify_path(path)
          if actual == "unknown" and detect_text_encoding(path) != "unknown":
              actual = "TXT" # infer text from bytes, regardless of suffix
          if claimed != actual:
              mismatches.append({"path": path,
                                 "claimed_type": claimed,
                                 "actual_type": actual})
  return mismatches
```

Made by Tha

Task 6: Find files that validly match multiple formats

6 · detect_polyglot(filepath) · worked solution

1. Load or stream the file once, then ask each supported format validator whether the bytes satisfy that format's structure. Add each valid type once.
2. For a ZIP-based file, verify the central directory and inspect members. A string like `%PDF-` somewhere inside a PNG is not, by itself, proof of a valid PDF; validate the PDF header position and its required ending structure too.
3. Return an empty list for no recognized valid format and a list of all valid matches otherwise. More than one match is the polyglot signal. Sort the result if deterministic output is required.
4. **The function.** Each entry of `FORMAT_VALIDATORS` maps a type name to a function that takes the file's bytes and returns whether they form a complete, valid file of that type:

```
def detect_polyglot(filepath):
  if isinstance(filepath, bytes):
      data = filepath
  else:
      with open(filepath, "rb") as f:
          data = f.read()
  matches = set()
  for kind, validator in FORMAT_VALIDATORS.items():
      if validator(data): # each validator checks the complete format structure
          matches.add(kind)
  return sorted(matches)
```

Made by Thanate A.

Task 7: Format an exact hexdump

7 · bytes_to_hexdump(data, width=16) · worked solution

1. For each offset 0, width, 2×width, …, slice at most width bytes. Format the offset as eight uppercase hex digits and each byte as two uppercase hex digits.
2. Pad the hex column to width × 3 characters. This makes the ASCII bar start in one column even when the last chunk is short.
3. Printable ASCII is byte values 0x20 through 0x7E inclusive; render every other value as a dot. Keep the separator, padding, and no-final-newline behavior exact.
4. **The whole function:**

```
def bytes_to_hexdump(data, width=16):
  if width <= 0:
      raise ValueError("width must be positive")
  lines = []
  for offset in range(0, len(data), width):
      chunk = data[offset:offset + width]
      hex_column = " ".join(f"{byte:02X}" for byte in chunk)
      hex_column = hex_column.ljust(width * 3)
      ascii_column = "".join(chr(b) if 0x20 <= b <= 0x7E else "."
                             for b in chunk)
      lines.append(f"{offset:08X} {hex_column} |{ascii_column}|")
  return "\n".join(lines)
```

Made by Thanate An

Task 8: Classify common text encodings from bytes

8 · detect_text_encoding(filepath) · worked solution

1. A UTF-8 BOM is EF BB BF; UTF-16 little-endian is FF FE; UTF-16 big-endian is FE FF. BOM checks take priority over heuristics.
2. Without a BOM, UTF-16 text often has many zero bytes in one parity: ASCII-heavy LE text has zeros at odd indexes, while BE has zeros at even indexes. This heuristic needs enough bytes to show the pattern; short no-BOM strings are ambiguous and should be reported as unknown when they contain zero bytes. Binary data can also imitate the pattern.
3. ASCII is a subset of UTF-8, so test all-ASCII bytes first when the result labels distinguish them. For non-ASCII bytes, try strict UTF-8 decoding explicitly; if it fails, use the assignment's Latin-1 fallback. This is explicit validation, not automatic encoding detection.
4. **The function,** written on bytes so Task 10 can reuse it; `detect_text_encoding(filepath)` only reads the file and passes the bytes on:

```
def detect_text_encoding(filepath):
  with open(filepath, "rb") as f:
      return detect_text_encoding_bytes(f.read())

def detect_text_encoding_bytes(data):
  if data.startswith(b"\xef\xbb\xbf"):
      return "UTF-8"
  if data.startswith(b"\xff\xfe"):
      return "UTF-16-LE"
  if data.startswith(b"\xfe\xff"):
      return "UTF-16-BE"
  paired = data[:len(data) - (len(data) % 2)]
  even_zeros = sum(paired[i] == 0 for i in range(0, len(paired), 2))
  odd_zeros = sum(paired[i] == 0 for i in range(1, len(paired), 2))
  if len(paired) >= 8 and odd_zeros > len(paired) // 4 and odd_zeros > even_zeros:
      return "UTF-16-LE"
  if len(paired) >= 8 and even_zeros > len(paired) // 4 and even_zeros > odd_zeros:
      return "UTF-16-BE"
  if b"\x00" in data:
      return "unknown" # short no-BOM UTF-16 and binary data are ambiguous
  controls = sum(byte < 0x20 and byte not in (9, 10, 12, 13) for byte in data)
  if data and controls > len(data) // 100:
      return "unknown" # reject mostly binary/control content
  if not data or all(byte < 0x80 for byte in data):
      return "ASCII"
  try:
      data.decode("utf-8", errors="strict")
      return "UTF-8"
  except UnicodeDecodeError:
      return "Latin-1"
```

Task 9: Find the first byte after the valid file structure

9 · find_appended_data(filepath, filetype) · worked solution

1. For PNG, start after its 8-byte signature. Each chunk is 4-byte big-endian length, 4-byte type, data, and 4-byte CRC; advance by 12 + length until a valid zero-length `IEND` chunk. The first byte after `IEND` is the candidate payload start.
2. For JPEG, find the EOI marker FF D9 after correctly handling marker segments and byte-stuffed scan data. For GIF, parse image/extension sub-blocks to the trailer byte. For PDF, use the final `%%EOF` marker and account for allowed trailing whitespace.
3. For ZIP, locate and validate the EOCD record; its 16-bit comment length determines the archive end. BMP stores its declared file size. RTF needs escaped-brace and binary-section handling; GZIP must distinguish concatenated members; ELF/SQLite derive the end from validated tables or page counts.
4. Return `None` when the structure is invalid, truncated, unsupported, or ends exactly at EOF. Marker search alone can match bytes inside payloads, so validate lengths and bounds before accepting an offset.
5. **The PNG branch.** Write the parser on bytes as well: `find_appended_data(filepath, filetype)` reads the file once and calls `find_appended_data_bytes(data, filetype)`, which runs the branch for that type (this is the PNG one):

```
# inside find_appended_data_bytes(data, filetype), when filetype == "PNG"
if not data.startswith(SIGNATURES["PNG"][0]):
  return None
pos = 8 # after the PNG signature
while pos + 12 <= len(data):
  length = int.from_bytes(data[pos:pos + 4], "big")
  kind = data[pos + 4:pos + 8]
  end = pos + 12 + length # header + data + CRC
  if end > len(data):
      return None # truncated chunk
  chunk = data[pos + 8:pos + 8 + length]
  stored_crc = int.from_bytes(data[pos + 8 + length:end], "big")
  if zlib.crc32(kind + chunk) & 0xFFFFFFFF != stored_crc:
      return None
  if kind == b"IEND":
      if length != 0:
          return None
      return None if end == len(data) else end
  pos = end
return None
```

Task 10: Aggregate the forensic report and write the personalized justification

10 · generate_forensics_report(root_dir) · worked solution

1. Walk every path once. Read that file's bytes once, then pass the in-memory bytes to the signature, encoding, hexdump, polyglot, and appended-data helpers. Hash the same bytes with `hashlib.sha256`; do not reopen the path for each task.
2. For every file, record its path, first bytes, detected type, extension claim/mismatch, checksum, encoding result, polyglot types, appended offsets, and any bounded hexdump preview. If no known signature matches but the bytes pass Task 8 text checks, call the actual type `TXT` independent of its suffix. Keep values to JSON-compatible strings, numbers, lists, dictionaries, booleans, and nulls.
3. Keep aggregate mismatches in their own list if that is the required schema. Benchmark with `time.perf_counter` on at least 500 files; report the measured elapsed time and environment rather than inventing a "within budget" result.
4. **The loop** (with `import hashlib, io, os, zipfile`), the body of `generate_forensics_report(root_dir)`:

```
report = {"mismatches": [], "files": []}
for folder, directories, names in os.walk(root_dir):
  directories.sort()
  for name in sorted(names):
      path = os.path.join(folder, name)
      with open(path, "rb") as f:
          data = f.read() # reuse these bytes; do not reopen for each task
      claimed = EXTENSION_TYPES.get(os.path.splitext(name)[1].lower(), "unknown")
      actual = identify_file_type(data[:16])
      if actual == "ZIP" and zipfile.is_zipfile(io.BytesIO(data)):
          actual = identify_zip_members(io.BytesIO(data))
      if actual == "unknown" and detect_text_encoding_bytes(data) != "unknown":
          actual = "TXT" # infer from bytes, never from the suffix
      types = detect_polyglot(data)
      appended = {kind: offset for kind in types
                  if (offset := find_appended_data_bytes(data, kind)) is not None}
      row = {"path": path, "magic_hex": data[:16].hex(),
            "actual_type": actual, "claimed_type": claimed,
            "sha256": hashlib.sha256(data).hexdigest(),
            "text_encoding": detect_text_encoding_bytes(data),
            "polyglot_types": types, "appended_offsets": appended,
            "hexdump_preview": bytes_to_hexdump(data[:64])}
      report["files"].append(row)
      if claimed != actual:
          report["mismatches"].append({"path": path,
              "claimed_type": claimed, "actual_type": actual})
return report
```

Task 10: Aggregate the forensic report and write the personalized justification

Written justification · use your own sample offsets · worked solution

1. For the polyglot, name the file from your samples directory, list every format your validators accepted, and identify the byte evidence for each. Explain why both structures are valid rather than citing two coincidental marker strings.
2. For the appended payload, report the file type, the end-marker or structural rule used, the returned byte offset, total file length, and number of trailing bytes (file length minus the offset). Include a short hexdump around the boundary.
3. The guide intentionally does not publish a fixed filename or offset: this assignment seeds personalized examples. Run your own report and paste your actual results into your submission.
4. **A starting point** for finding your two rows:

```
# Use your own generated files; never copy another learner's offsets.
print(report["files"]) # select the row with multiple valid types
# Then compare appended_offsets[type] with that file's byte length.
```

Made by Thanate Amornjiraporn

Personal worked-solution guide based on the APDR Computer Exercise 1 task sheet, rewritten in my own words. Its ZIP-family requirement cannot be satisfied from the first 16 bytes alone; the page separates signature identification from archive inspection. The written justification must use each learner's own seeded sample offsets. Not an official FAU publication. Made by **Thanate Amornjiraporn (Focus)**.